where the list can show either active or inactive clients. We can add a route which captures the `:status` parameter in a "pretty" URL:

```
get '/clients/:status' => 'clients#index', foo: 'bar'
```

In this case, when a user opens the URL `/clients/active`, `params[:status]` will be set to "active". When this route is used, `params[:foo]` will also be set to "bar" just like it was passed in the query string. In the same way `params[:action]` will contain "index".

4.4 `default_url_options`

You can set global default parameters for URL generation by defining a method called `default_url_options` in your controller. Such a method must return a hash with the desired defaults, whose keys must be symbols:

```
class ApplicationController < ActionController::Base
  def default_url_options
    { locale: I18n.locale }
  end
end
```

These options will be used as a starting point when generating URLs, so it's possible they'll be overridden by the options passed in `url_for` calls.

If you define `default_url_options` in `ApplicationController`, as in the example above, it would be used for all URL generation. The method can also be defined in one specific controller, in which case it only affects URLs generated there.

4.5 Strong Parameters

With strong parameters, Action Controller parameters are forbidden to be used in Active Model mass assignments until they have been whitelisted. This means you'll have to make a conscious choice about which attributes to allow for mass updating and thus prevent accidentally exposing that which shouldn't be exposed.

In addition, parameters can be marked as required and flow through a predefined raise/rescue flow to end up as a 400 Bad Request with no effort.

```
class PeopleController < ActionController::Base
  # This will raise an ActiveModel::ForbiddenAttributes exception
  # because it's using mass assignment without an explicit permit
  # step.
  def create
    Person.create(params[:person])
  end

  # This will pass with flying colors as long as there's a person key
  # in the parameters, otherwise it'll raise a
  # ActionController::ParameterMissing exception, which will get
  # caught by ActionController::Base and turned into that 400 Bad
  # Request reply.
  def update
    person = current_account.people.find(params[:id])
    person.update!(person_params)
    redirect_to person
  end

  private
    # Using a private method to encapsulate the permissible parameters
    # is just a good pattern since you'll be able to reuse the same
    # permit list between create and update. Also, you can specialize
    # this method with per-user checking of permissible attributes.
```